Phase 3, days 61 to 90: Prove

EY: Talent Data Strategy

B5 Data validation

B5-L1 DataFrame schemas with pandera

A schema at the point of entry turns a silent bad input into one loud, detailed failure.

```python
import pandera.pandas as pa

RESPONSES = pa.DataFrameSchema(
    {
        "respondent_id": pa.Column(str, unique=True),
        "age": pa.Column(int, pa.Check.in_range(16, 99)),
        "weight": pa.Column(float, pa.Check.gt(0)),
    },
    strict=True,
    coerce=True,
)
clean = RESPONSES.validate(raw, lazy=True)
```

pandera Column defaults	nullable=False. Set nullable=True where nulls are expected.
strict=True and coerce=True	strict rejects unknown columns. coerce converts dtypes before checks.
validate(df, lazy=True)	Runs every check and raises SchemaErrors with failure_cases.
SchemaError vs SchemaErrors	Separate classes. SchemaErrors is not a subclass. Catch both, or better, let them raise.

B5-L2 pydantic models for config and input records

Validate external settings and records once, at the boundary, with every rule in the model.

```python
from pydantic import BaseModel, ConfigDict, Field


class WeightingConfig(BaseModel):
    model_config = ConfigDict(extra="forbid", frozen=True)

    weight_col: str
    min_base: int = Field(default=50, gt=0)
    cap: float = Field(default=5.0, gt=1.0)


cfg = WeightingConfig.model_validate(yaml.safe_load(text))
```

pydantic extra keys, default behaviour	Ignored silently. Use ConfigDict(extra="forbid") for config.
pydantic lax mode	Converts "30" and 30.0 to 30, True to 1. Use strict=True to refuse conversions.
model_copy(update=...)	Does not validate. Rebuild with model_validate({**m.model_dump(), ...}).
pydantic or pandera	pydantic for records and settings from outside. pandera for whole DataFrames.

B5-L3 Validate at the boundaries, with severity

Check at boundaries. Collect structured results. Log them all, then stop on CRITICAL.

```python
Severity = Literal["CRITICAL", "WARN"]


@dataclass(frozen=True)
class QaResult:
    check: str
    severity: Severity
    passed: bool
    observed: float | None
```

Where to validate	At boundaries: after reading input, before writing output.
CRITICAL vs WARN	CRITICAL stops the run (raise). WARN records and continues.
Order of QA operations	Run every check, log all results, then raise if any CRITICAL failed.
Two coding rules for checks	Guard every division. Test for None with is None, never truthiness.

A10 Deployment

A10-L1 Git folders, one branch per change, and no secrets in the repository

Code flows from a branch, through review and CI, to production. Secrets flow from a secret store at run time, never through Git.

```python
api_key = dbutils.secrets.get(scope="research", key="supplier_api_key")
```

Where do developers work in Databricks Git folders?	Their own clone under /Workspace/Users/<email>/, on their own branch.
Who updates production Git folders?	Only automation, after a pull request merges into the deployment branch.
Read a secret in a notebook	dbutils.secrets.get(scope="...", key="..."). Output shows [REDACTED].
A token was committed, then removed. Is it safe?	No. Git history keeps it. Rotate the token.

A10-L2 Declarative Automation Bundles and parameterised jobs

One reviewed file describes the job for every environment. The target decides where it writes.

```yaml
bundle:
  name: research_pipelines

variables:
  catalog:
    description: Catalog the pipelines write to.

resources:
  jobs:
    nat_silver_pipeline:
      name: nat_silver_pipeline
      parameters:
        - name: catalog
```

```
          default: ${var.catalog}
      tasks:
        - task_key: silver
          notebook_task:
            notebook_path: ./notebooks/silver_nat_transform.py

targets:
  dev:
    mode: development
    default: true
    variables:
      catalog: research_dev
  prod:
    mode: production
    variables:
      catalog: research_prod
```

```
databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle run -t dev nat_silver_pipeline
```

What are Declarative Automation Bundles?	Jobs and their settings as YAML in Git (formerly Databricks Asset Bundles). Same CLI: databricks bundle.
Targets and variables	Targets are environments. Variables hold what differs, such as the catalog, set per target.
What does mode: development do?	Prefixes resources with [dev <user>] and pauses schedules and triggers.
Read a job parameter in a notebook	dbutils.widgets.get("name"). Validate it before using it.

A10-L3 CI: lint, type-check, test and validate on every pull request

Every pull request is linted, typed, tested and validated. Only a merge to main deploys, as a service principal, with nothing secret in the file. The CLI version shown is an example: pin the version you tested.

```
env:
  DATABRICKS_HOST: ${{ vars.DATABRICKS_HOST }}
  DATABRICKS_CLIENT_ID: ${{ secrets.DATABRICKS_CLIENT_ID }}
  DATABRICKS_CLIENT_SECRET: ${{ secrets.DATABRICKS_CLIENT_SECRET }}
```

Four CI checks for a pipeline repository	ruff (lint and format), mypy, pytest with local Spark, databricks bundle validate.
Who deploys to production?	CI, after a merge to main, as a service principal.
OAuth M2M environment variables	DATABRICKS_HOST, DATABRICKS_CLIENT_ID, DATABRICKS_CLIENT_SECRET from the CI secret store.
Two ways CI lies	continue-on-error: true, and command || true. Both turn failures green.

B13 Interpretability and fairness

B13-L1 Permutation importance on held-out data

Measure importance by shuffling on held-out data, with the reported metric, and report the spread.

```
from sklearn.inspection import permutation_importance
result = permutation_importance(model, X_test, y_test, scoring="roc_auc",
```

```
                                n_repeats=10, random_state=0)
result.importances_mean, result.importances_std
```

Impurity importance bias	Favours high-cardinality features, from training data. A random ID scored 0.335 in this lesson's example.
Permutation importance	Shuffle a column on held-out data and measure the score drop, with its spread.
Correlated features	Share credit. Permute and report them as a group.
What importance means	What the model relies on. Not what causes the outcome.

B13-L2 SHAP values, and what they do not tell you

SHAP explains one prediction additively in a stated output space. It describes the model, not the world.

```
prediction = base_value + sum of the row's SHAP values
```

```
import shap
explainer = shap.TreeExplainer(model)
explanation = explainer(X_test)          # shap.Explanation
explanation.values, explanation.base_values
```

SHAP additivity	prediction = base_value + sum of the row's SHAP values, in the explainer's output space.
Output space for GradientBoostingClassifier in TreeExplainer	Log-odds (decision_function), not probability.
Global ranking from SHAP	Mean absolute SHAP value per feature.
What SHAP cannot show	Causes, a stable split between correlated features, behaviour outside the training range.

B13-L3 Performance by subgroup, and the fairness trade-offs

Measure rates per group with counts, choose the fairness property that fits the decision, and document the gaps.

Rates to report per group	n, positives, selection rate, TPR, FPR, precision, calibration.
Impossibility result	With different base rates, calibration and equal error rates cannot all hold (Kleinberg et al. 2016; Chouldechova 2017).
Dropping a protected attribute	Proxies keep its influence. Keep it for measurement where lawful.
Choosing a fairness property	From the harm: missed help (TPR), wrong flags (FPR), numbers used directly (calibration).

B12 Model evaluation

B12-L1 Choose the metric from the decision

Choose metrics and thresholds from the decision. Report prevalence and counts beside every number.

Accuracy at 2.4% prevalence	0.976 for predicting all negatives. Useless as a headline.
Baseline for average precision	The prevalence. Report it beside the score.

Where the threshold comes from	The decision: review capacity or costs. Not the 0.5 default.
Precision and recall	Precision: share of flags that are real. Recall: share of real cases flagged.

B12-L2 Calibration, when the probability itself matters

When the probability is used as a number, check calibration, compare the Brier score with a constant baseline, and recalibrate on held-out data.

Calibrated means	Of cases scored p, about p are positive.
Check calibration	Calibration table or calibration_curve, plus Brier score against the constant prevalence baseline.
class_weight="balanced" and probabilities	Pushes them towards 0.5. Mean p went from 0.024 to 0.42 at 2.4% prevalence.
Recalibrate	CalibratedClassifierCV(model, method="sigmoid", cv=5). Isotonic only with plenty of data.

B12-L3 Imbalanced classes and error analysis by segment

Treat rare positives with a chosen threshold first, then break results down by segment and read the worst errors.

First response to rare positives	Train normally and choose the threshold from capacity or cost.
Resampling rule	Only inside training folds. Never the test set.
Error analysis	Metrics per segment with counts, then read the most confident errors.
Pooled vs segment recall	Pooling weights large segments. Overall 0.7 can hide a segment at 0.2.